

```

1:
2: import pandas as pd
3: import matplotlib.pyplot as plt
4: import seaborn as sns
5: import numpy as np
6:
7:
8: #creating the dataframes
9: df = pd.read_csv('qvi_data.csv')
10: df.columns = df.columns.str.lower() # making the column names all lowercase
11: df['date'] = pd.to_datetime(df['date']) # changing the column datatype
12: df['month'] = df['date'].dt.to_period('M') # creating a column for month
13:
14: # --- Creating some metrics to analyse the stores behavior ---
15: revenue = df.groupby(['store_nbr', 'month'])['tot_sales'].sum().reset_index(name='monthly_revenue')
16: customers = df.groupby(['store_nbr', 'month'])['lylty_card_nbr'].count().reset_index(name='monthly_customers')
17: transactions = df.groupby(['store_nbr', 'month'])['txn_id'].count().reset_index(name='monthly_txns')
18: store_counts = revenue['store_nbr'].value_counts() # checking how many months that store made sells
19:
20: #print(store_counts.head(20))
21: full_months = store_counts[store_counts == 12].index # filtering only stores with revenue during all period
22: df_filtered = revenue[revenue['store_nbr'].isin(full_months)]
23: monthly_metrics = revenue.merge(customers, on=['store_nbr', 'month']).merge(transactions, on=['store_nbr', 'month'])
24: '''
25:     store_nbr    month  montly_revenue  monthly_customers  monthly_txns
26: 0           1  2018-07             206.9                52           52
27: 1           1  2018-08             176.1                43           43
28: 2           1  2018-09             278.8                62           62
29: 3           1  2018-10             188.1                45           45
30: 4           1  2018-11             192.6                47           47
31: '''
32: monthly_metrics = monthly_metrics[monthly_metrics['store_nbr'].isin(full_months)] # this new merged df will show only the stores that follow
the role: sales in all twelve months
33:
34: #print(monthly_metrics.info())
35: '''
36:     <class 'pandas.DataFrame'>
37: Index: 3120 entries, 0 to 3168
38: Data columns (total 5 columns):
39:  #   Column                Non-Null Count  Dtype
40: ---  ---
41: 0   store_nbr             3120 non-null    int64
42: 1   month                 3120 non-null    period[M]
43: 2   montly_revenue        3120 non-null    float64
44: 3   monthly_customers     3120 non-null    int64
45: 4   monthly_txns          3120 non-null    int64
46: dtypes: float64(1), int64(3), period[M](1)
47: '''
48:
49: # The period before the tryal layout is between Jul 2018 and Jan 2019
50: before_trial = monthly_metrics[monthly_metrics['month'] < '2019-02']
51: before_trial = before_trial.sort_values(by='month')
52: '''
53: How about to use Streamlit to create a website where the user can analyse each trial store metrics over each control candidate store metrics
?
54: In that case, the user can choose the trial store, and the grafics will show the metrics before and after the trial period.
55:
56: '''
57: before_trial_pivot = before_trial.pivot(index='store_nbr', columns='month', values=['monthly_revenue', 'monthly_customers', 'monthly_txns'])
58:
59: '''

```

```

60:     monthly_revenue
61: month      2018-07  2018-08 2018-09 2018-10 2018-11 2018-12 2019-01
62: store_nbr
63: 1          206.9   176.10   278.8   188.1   192.6   189.6   154.8
64: 2          150.8   193.80   154.4   167.8   162.9   136.0   162.8
65: 3          1205.7  1079.75  1021.5  1037.9  1008.0  1121.6  1051.7
66:
67:     monthly_customers
68: month      2018-07 2018-08 2018-09 2018-10 2018-11 2018-12 2019-01
69: store_nbr
70: 1          52.0    43.0    62.0    45.0    47.0    47.0    36.0
71: 2          41.0    43.0    37.0    43.0    40.0    38.0    45.0
72: 3          138.0   134.0   119.0   119.0   118.0   129.0   121.0
73:
74:     monthly_txns
75: month      2018-07 2018-08 2018-09 2018-10 2018-11 2018-12 2019-01
76: store_nbr
77: 1          52.0    43.0    62.0    45.0    47.0    47.0    36.0
78: 2          41.0    43.0    37.0    43.0    40.0    38.0    45.0
79: 3          138.0   134.0   119.0   119.0   118.0   129.0   121.0
80: '''
81:
82: # Correlation tests
83: trial_stores_nbr = [77, 86, 88] # a list of the trial stores
84: control_candidates = before_trial[~before_trial['store_nbr'].isin(trial_stores_nbr)] # a new nf with only the stores candidates to be the co
ntrol
85: trial_stores = before_trial[before_trial['store_nbr'].isin(trial_stores_nbr)] # a new df with only the three trial stores
86:
87: # extracting the Series for two different stores
88: series_candidate = control_candidates[control_candidates['store_nbr'] == 1]['monthly_revenue']
89: series_trial = trial_stores[trial_stores['store_nbr'] == 77]['monthly_revenue']
90:
91: # calculating the correlation (returns a number between -1 and 1)
92: correlation = series_trial.reset_index(drop=True).corr(series_candidate.reset_index(drop=True), method='pearson')
93: # calculating the average absolute distance
94: distance = np.mean(np.abs(series_trial.values - series_candidate.values))
95:
96: # --- function to returns a list of complete dictionaries containing a store number, correlation, raw distance, scaled distance and composi
e score ---
97: '''
98: def store_search(store, df, metric):
99:     series_trial = df[df['store_nbr'] == store][metric].reset_index(drop=True)
100:     results = []
101:     all_stores = df['store_nbr'].unique()
102:     for candidate in all_stores:
103:         if candidate == store:
104:             continue
105:
106:         series_candidate = df[df['store_nbr'] == candidate][metric].reset_index(drop=True)
107:
108:         raw_corr = series_trial.corr(series_candidate, method='pearson')
109:
110:         raw_dist = np.mean(np.abs(series_trial.values - series_candidate.values))
111:
112:         results.append({
113:             'store_nbr': candidate,
114:             'correlation': raw_corr,
115:             'raw_distance': raw_dist
116:         })
117:
118:     distances = []

```

```
119:     for item in results:
120:         distances.append(item['raw_distance'])
121:
122:     min_dist = min(distances)
123:     max_dist = max(distances)
124:
125:     for item in results:
126:         # calculating the scaled distance
127:         scaled_dist = 1 - ((item['raw_distance'] - min_dist) / (max_dist - min_dist))
128:
129:         # adding scaled_distance to the item dictionary
130:         item['scaled_distance'] = scaled_dist
131:
132:         # calculating the composite score:
133:         composite = (item['correlation'] + scaled_dist) / 2
134:
135:         # adding composite_score to the item dictionary
136:         item['composite_score'] = composite
137:
138:     return results
139:
140:
141: # function to display the previous result clearly
142:
143: def get_top_matches(results_list):
144:     # creating a df from the list of dictionaries
145:     df_results = pd.DataFrame(results_list)
146:
147:     # sorting the df by 'composite_score' in descending order, so the highest score/best match is at the very top
148:     df_sorted = df_results.sort_values(by='composite_score', ascending=False)
149:
150:     # returning the sorted df
151:     return df_sorted.head(10)
152:
153: # metrics to be visualized
154: metrics = ['monthly_revenue', 'monthly_customers', 'monthly_txns']
155:
156: # trial loop
157: for trial in [77, 86, 88]:
158:
159:     print(f"\n Finding the best Control Store for Trial Store {trial}")
160:
161:     metric_dfs = []
162:
163:     for metric in metrics:
164:         # getting raw list of results for the specific metric
165:
166:         raw_results = store_search(store=trial, df=before_trial, metric=metric)
167:
168:         # converting to a DataFrame
169:         df_metric = pd.DataFrame(raw_results)
170:
171:         # Keeping only the store_nbr and the composite_score for this metric
172:         # renaming the composite_score column so we know which metric it belongs to
173:         df_metric = df_metric[['store_nbr', 'composite_score']].rename(columns={'composite_score': f'score_{metric}'})
174:
175:         metric_dfs.append(df_metric)
176:
177:     # Merging the three metric dataframes together on store_nbr
178:     final_df = metric_dfs[0]
179:     for df_m in metric_dfs[1:]:
```

```
180:     final_df = pd.merge(final_df, df_m, on='store_nbr')
181:
182:     # calculating the ultimate score across all metrics
183:     score_columns = [f'score_{m}' for m in metrics]
184:     final_df['ultimate_score'] = final_df[score_columns].mean(axis=1)
185:
186:     # sort and display the top matches
187:     final_df = final_df.sort_values(by='ultimate_score', ascending=False)
188:     #print(final_df.head(5))
189: '''
190: # ---
191: '''
192: Finding the best Control Store for Trial Store 77
193:     store_nbr  score_monthly_revenue  score_monthly_customers  score_monthly_txns  ultimate_score
194: 175      233              0.951887              0.982320              0.982320              0.972176
195: 212      41              0.880579              0.936174              0.936174              0.917642
196: 50       17              0.864397              0.911242              0.911242              0.895627
197: 14       254             0.754326              0.925547              0.925547              0.868473
198: 67       115             0.815929              0.892925              0.892925              0.867260
199:
200: Finding the best Control Store for Trial Store 86
201:     store_nbr  score_monthly_revenue  score_monthly_customers  score_monthly_txns  ultimate_score
202: 92      138              0.859483              0.916353              0.916353              0.897396
203: 103     155              0.938423              0.826063              0.826063              0.863517
204: 16      147              0.744843              0.877096              0.877096              0.833011
205: 200     166              0.744243              0.866033              0.866033              0.825436
206: 151     247              0.752913              0.834398              0.834398              0.807236
207:
208: Finding the best Control Store for Trial Store 88
209:     store_nbr  score_monthly_revenue  score_monthly_customers  score_monthly_txns  ultimate_score
210: 10      7              0.731151              0.883068              0.883068              0.832429
211: 35     123              0.647417              0.909730              0.909730              0.822292
212: 147    237              0.654240              0.838279              0.838279              0.776932
213: 124    178              0.728884              0.787164              0.787164              0.767737
214: 232    203              0.750972              0.750969              0.750969              0.750970
215: '''
216:
217: # --- Trial period
218:
219: trial_period = monthly_metrics[monthly_metrics['month'].between('2019-02', '2019-04')]
220:
221: trial_period = trial_period.sort_values(by='month')
222:
223: #print(trial_period.sample(5))
224: '''
225:     store_nbr  month  monthly_revenue  monthly_customers  monthly_txns
226: 261      23  2019-02              867.8              127              127
227: 2297     197  2019-04              262.2              42              42
228: 2550     220  2019-02              202.9              43              43
229: 1237     108  2019-02              321.4              41              41
230: 575      50  2019-03              227.0              45              45
231: '''
232:
233: # mapping trial stores to their absolute best multi-metric control matches
234: control_mapping = {
235:     77: 233,
236:     86: 138,
237:     88: 7
238: }
239:
240: trial_results = []
```

```

241:
242: for k, v in control_mapping.items():
243:     metrics = ['monthly_revenue', 'monthly_customers', 'monthly_txns']
244:
245:     for metric in metrics:
246:         # Calculating the scaling factor:
247:         pre_trial = before_trial[before_trial['store_nbr'] == k][metric].sum()
248:
249:         pre_control = before_trial[before_trial['store_nbr'] == v][metric].sum()
250:
251:         scaling_factor = pre_trial / pre_control
252:
253:         # getting the unique months available in the trial period
254:         trial_months = trial_period['month'].unique()
255:
256:         for month in trial_months:
257:             # isolating the row for the trial store and control store for this specific month
258:             trial_row = trial_period[(trial_period['store_nbr'] == k) & (trial_period['month'] == month)]
259:             control_row = trial_period[(trial_period['store_nbr'] == v) & (trial_period['month'] == month)]
260:
261:             # checking to make sure rows exists for both stores in this month
262:             if not trial_row.empty and not control_row.empty:
263:                 actual_trial_val = trial_row[metric].values[0]
264:                 actual_control_val = control_row[metric].values[0]
265:
266:                 # applying the scaling factor to the control value
267:                 scaled_control_val = actual_control_val * scaling_factor
268:
269:                 # calculating percentage uplift
270:                 percentage_diff = ((actual_trial_val - scaled_control_val) / scaled_control_val) * 100
271:
272:                 # saving everything to our results list
273:                 trial_results.append({
274:                     'trial_store': k,
275:                     'control_store': v,
276:                     'metric': metric,
277:                     'month': month,
278:                     'trial_actual': actual_trial_val,
279:                     'control_scaled': scaled_control_val,
280:                     'uplift_pct': percentage_diff
281:                 })
282:
283: df_uplift = pd.DataFrame(trial_results)
284:
285: #print(f"\n{'==' * 18}Trial period uplift analysis{'==' * 18}")
286:
287: #print(df_uplift.to_string())
288: '''
289: =====Trial period uplift analysis=====
290:   trial_store  control_store      metric  month  trial_actual  control_scaled  uplift_pct
291: 0            77            233  monthly_revenue  2019-02         235.0         249.762622    -5.910661
292: 1            77            233  monthly_revenue  2019-03         278.5         203.802205    36.652103
293: 2            77            233  monthly_revenue  2019-04         263.5         162.345704    62.307960
294: 3            77            233  monthly_customers  2019-02          45.0         47.906752    -6.067521
295: 4            77            233  monthly_customers  2019-03          55.0         41.790997    31.607294
296: 5            77            233  monthly_customers  2019-04          48.0         33.636656    42.701463
297: 6            77            233    monthly_txns  2019-02          45.0         47.906752    -6.067521
298: 7            77            233    monthly_txns  2019-03          55.0         41.790997    31.607294
299: 8            77            233    monthly_txns  2019-04          48.0         33.636656    42.701463
300: 9            86            138  monthly_revenue  2019-02          913.2         724.548428    26.037124
301: 10           86            138  monthly_revenue  2019-03         1026.8         910.379711    12.788102

```

```

302: 11      86      138      monthly_revenue 2019-04      848.2      807.398208      5.053491
303: 12      86      138      monthly_customers 2019-02      139.0      107.960656      28.750607
304: 13      86      138      monthly_customers 2019-03      142.0      133.022951      6.748497
305: 14      86      138      monthly_customers 2019-04      127.0      123.383607      2.931016
306: 15      86      138      monthly_txns 2019-02      139.0      107.960656      28.750607
307: 16      86      138      monthly_txns 2019-03      142.0      133.022951      6.748497
308: 17      86      138      monthly_txns 2019-04      127.0      123.383607      2.931016
309: 18      88       7      monthly_revenue 2019-02      1370.2      1316.924460      4.045451
310: 19      88       7      monthly_revenue 2019-03      1477.2      1470.120856      0.481535
311: 20      88       7      monthly_revenue 2019-04      1439.4      1209.148101      19.042489
312: 21      88       7      monthly_customers 2019-02      154.0      152.058072      1.277096
313: 22      88       7      monthly_customers 2019-03      170.0      174.678281      -2.678227
314: 23      88       7      monthly_customers 2019-04      162.0      136.977933      18.267225
315: 24      88       7      monthly_txns 2019-02      154.0      152.058072      1.277096
316: 25      88       7      monthly_txns 2019-03      170.0      174.678281      -2.678227
317: 26      88       7      monthly_txns 2019-04      162.0      136.977933      18.267225
318: '''
319:
320: # charts building
321:
322: # seting aesthetics for clean presentation charts
323: sns.set_theme(style="whitegrid")
324: plt.rcParams.update({'font.size': 12})
325:
326: # Isolating pair data across the entire timeline
327: pair_77_233 = monthly_metrics[monthly_metrics['store_nbr'].isin([77, 233]).copy()]
328:
329: # ensuring month is formatted nicely for axis plotting
330: pair_77_233['month_str'] = pair_77_233['month'].astype(str)
331:
332: # computing pre-trial scaling factors
333: pre_trial_data = pair_77_233[pair_77_233['month'] < '2019-02']
334:
335: metrics = ['monthly_revenue', 'monthly_customers', 'monthly_txns']
336: scaling_factors = {}
337:
338: for m in metrics:
339:     trial_sum = pre_trial_data[pre_trial_data['store_nbr'] == 77][m].sum()
340:     control_sum = pre_trial_data[pre_trial_data['store_nbr'] == 233][m].sum()
341:     scaling_factors[m] = trial_sum / control_sum
342:
343: # creating a clean, scaled timeline dataframe
344: df_77 = pair_77_233[pair_77_233['store_nbr'] == 77][['month_str', 'monthly_revenue', 'monthly_customers', 'monthly_txns']].copy()
345:
346: df_233 = pair_77_233[pair_77_233['store_nbr'] == 233][['month_str', 'monthly_revenue', 'monthly_customers', 'monthly_txns']].copy()
347:
348: #applying scaling to control store 233
349: for m in metrics:
350:     df_233[f'{m}_scaled'] = df_233[m] * scaling_factors[m]
351:
352: #merging into one flat dataframe for easy plotting
353: df_plot_77 = pd.merge(
354:     df_77,
355:     df_233[['month_str', 'monthly_revenue_scaled', 'monthly_customers_scaled', 'monthly_txns_scaled']],
356: ).sort_values(by='month_str')
357:
358: '''
359: === Unified timeline dataframe (store 77 vs 233) ===
360: month_str monthly_revenue monthly_customers monthly_txns monthly_revenue_scaled monthly_customers_scaled monthly_txns_scaled
361: 0 2018-07      296.8      55      55      297.565550      55.041801      55.041801
362: 1 2018-08      255.5      48      48      292.652187      50.964630      50.964630

```

```

363: 2    2018-09    225.2    44    44    233.998916    45.868167    45.868167
364: 3    2018-10    204.5    38    38    190.085733    36.694534    36.694534
365: 4    2018-11    245.3    44    44    216.597421    41.790997    41.790997
366: '''
367:
368:
369: def pair_chart_generate(trial_store, control_store, monthly_metrics, before_trial):
370:     # 1. Calculate scaling factors
371:     rev_factor = before_trial[before_trial['store_nbr'] == trial_store]['monthly_revenue'].sum() / \
372:         before_trial[before_trial['store_nbr'] == control_store]['monthly_revenue'].sum()
373:
374:     cust_factor = before_trial[before_trial['store_nbr'] == trial_store]['monthly_customers'].sum() / \
375:         before_trial[before_trial['store_nbr'] == control_store]['monthly_customers'].sum()
376:
377:     txns_factor = before_trial[before_trial['store_nbr'] == trial_store]['monthly_txns'].sum() / \
378:         before_trial[before_trial['store_nbr'] == control_store]['monthly_txns'].sum()
379:
380:     # 2. Isolate and scale
381:     t_df = monthly_metrics[monthly_metrics['store_nbr'] == trial_store].copy()
382:     c_df = monthly_metrics[monthly_metrics['store_nbr'] == control_store].copy()
383:
384:     t_df['month_str'] = t_df['month'].astype(str)
385:     c_df['month_str'] = c_df['month'].astype(str)
386:
387:     c_df['monthly_revenue_scaled'] = c_df['monthly_revenue'] * rev_factor
388:     c_df['monthly_customers_scaled'] = c_df['monthly_customers'] * cust_factor
389:     c_df['monthly_txns_scaled'] = c_df['monthly_txns'] * txns_factor
390:
391:     df_plot = pd.merge(
392:         t_df[['month_str', 'monthly_revenue', 'monthly_customers', 'monthly_txns']],
393:         c_df[['month_str', 'monthly_revenue_scaled', 'monthly_customers_scaled', 'monthly_txns_scaled']],
394:         on='month_str'
395:     ).sort_values(by='month_str')
396:
397:     # 3. Generate the 3 charts
398:     chart_configs = [
399:         ('monthly_revenue', 'monthly_revenue_scaled', 'Monthly Revenue ($)', f'revenue_{trial_store}_vs_{control_store}.png'),
400:         ('monthly_customers', 'monthly_customers_scaled', 'Monthly Customers', f'customers_{trial_store}_vs_{control_store}.png'),
401:         ('monthly_txns', 'monthly_txns_scaled', 'Monthly Transactions', f'transactions_{trial_store}_vs_{control_store}.png')
402:     ]
403:
404:     for actual_col, scaled_col, title_label, file_name in chart_configs:
405:         plt.figure(figsize=(10, 5))
406:         plt.plot(df_plot['month_str'], df_plot[actual_col], marker='o', linewidth=2.5, color='#1f77b4', label=f'Trial Store {trial_store}')
407:         plt.plot(df_plot['month_str'], df_plot[scaled_col], marker='s', linestyle='--', linewidth=2, color='#ff7f0e', label=f'Control Store {control_store} (Scaled)')
408:         plt.axvline(x='2019-02', color='red', linestyle=':', linewidth=2, label='Trial Start (Feb 2019)')
409:
410:         plt.title(f'Store {trial_store} vs Control Store {control_store}: {title_label}', fontsize=14, fontweight='bold', pad=15)
411:         plt.xlabel('Month', fontsize=11, labelpad=10)
412:         plt.ylabel(title_label, fontsize=11)
413:         plt.xticks(rotation=45)
414:         plt.legend(loc='upper left', frameon=True)
415:         plt.tight_layout()
416:         plt.savefig(file_name, dpi=300, bbox_inches='tight')
417:         plt.close()
418:
419:     print(f"Generated all charts for Pair ({trial_store}, {control_store})")
420:
421: pairs = [
422:     (77, 233),

```

```
423:     (86, 138),
424:     (88, 7)
425: ]
426:
427: for trial, control in pairs:
428:     pair_chart_generate(trial, control, monthly_metrics, before_trial)
429:
430:
431:
```